

Lecture2: ECE C147A/C247A, Neural Networks & Deep Learning

Review of machine learning fundamentals: linear regression and classification

September 30th, 2026

These slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge.

Announcements and review of previous lecture

ANNOUNCEMENTS

- HW #1 posted October 3rd 2026 (Friday), 11.59 pm → note even 1 minute late is equal to 1 DAY late
  - ↳ see rules handout for late HWs etc.
- PTEs: cannot give now unless slots open up → encourage students to attend lectures till slots open up.
  - ↳ will try to put material on gradescope till then (2-3 weeks) - you can register on gradescope
- Discussion sessions: One in-person session Fridays 11am, will send Zoom link and post recordings
  - MENG students please attend this discussion section and we will also provide recordings
- Make up lecture - Friday, 12noon - 1.30pm, October 2nd on Zoom and will post recording → will send Zoom link by email to class soon
  - Wednesday October 7th lecture converted to a review session
- OFFICE HOURS: Please look at organizational handout #1 for times & locations
- Reminders - Please read rules handout HO#3 carefully as they will be strictly implemented
  - EXAM DATES ANNOUNCED → no alternate exams



REVIEW OF PREVIOUS LECTURE

- High level view of course & motivations for AI

- Supervised learning labeled data $\mathcal{D} = \{ (\underline{x}^{(1)}, y^{(1)}), \ldots, (\underline{x}^{(N)}, y^{(N)}) \}$

  N = # of labeled examples

  $\underline{x}^{(i)}$ → inputs

  $y^{(i)}$ → labels

  $\mathcal{D}$ ↓ dataset

  Using labels do prediction: first case is for continuous labels $y^{(i)} \in \mathbb{R}$ and learn function $f(\cdot)$

$$f(x^{TEST}) = \hat{y}^{TEST}$$

x^{TEST} → new sample

$\hat{y}^{TEST}$ → predict label

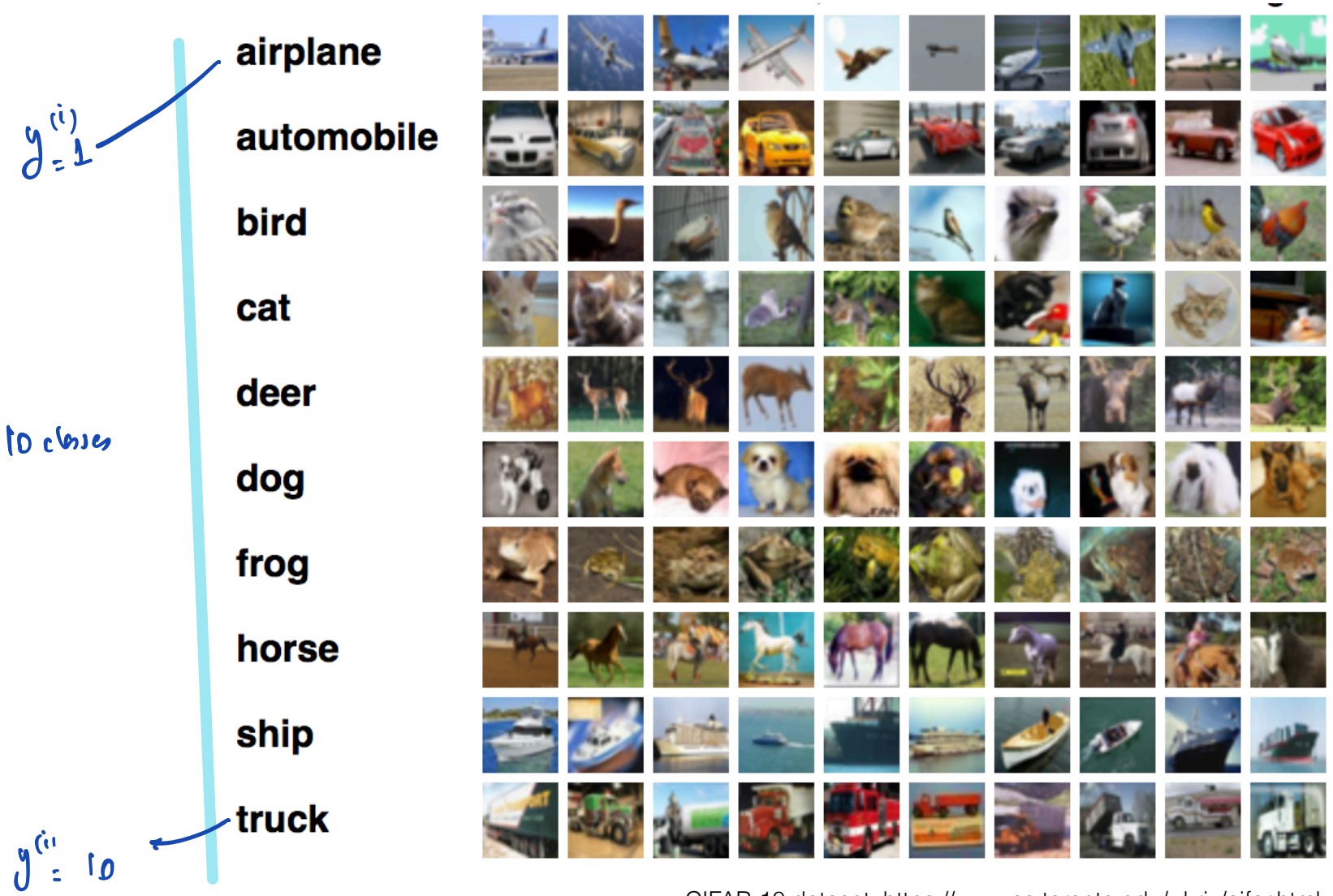
$f(\cdot)$ → to learn

- Goal: to find $f(\cdot)$ based on data $\mathcal{D}$

- Question: What criteria to use for designing $f(\cdot)$

Classification

CIFAR-10 dataset



CIFAR-10 dataset, https://www.cs.toronto.edu/~kriz/cifar.html

color

R, G, B

$x^{(i)}$ $32 \times 32 \times 3$

pixel sizes

$\underline{x} \in \mathbb{R}^{3072}$

$32 \times 32 \times 3 = 3072$

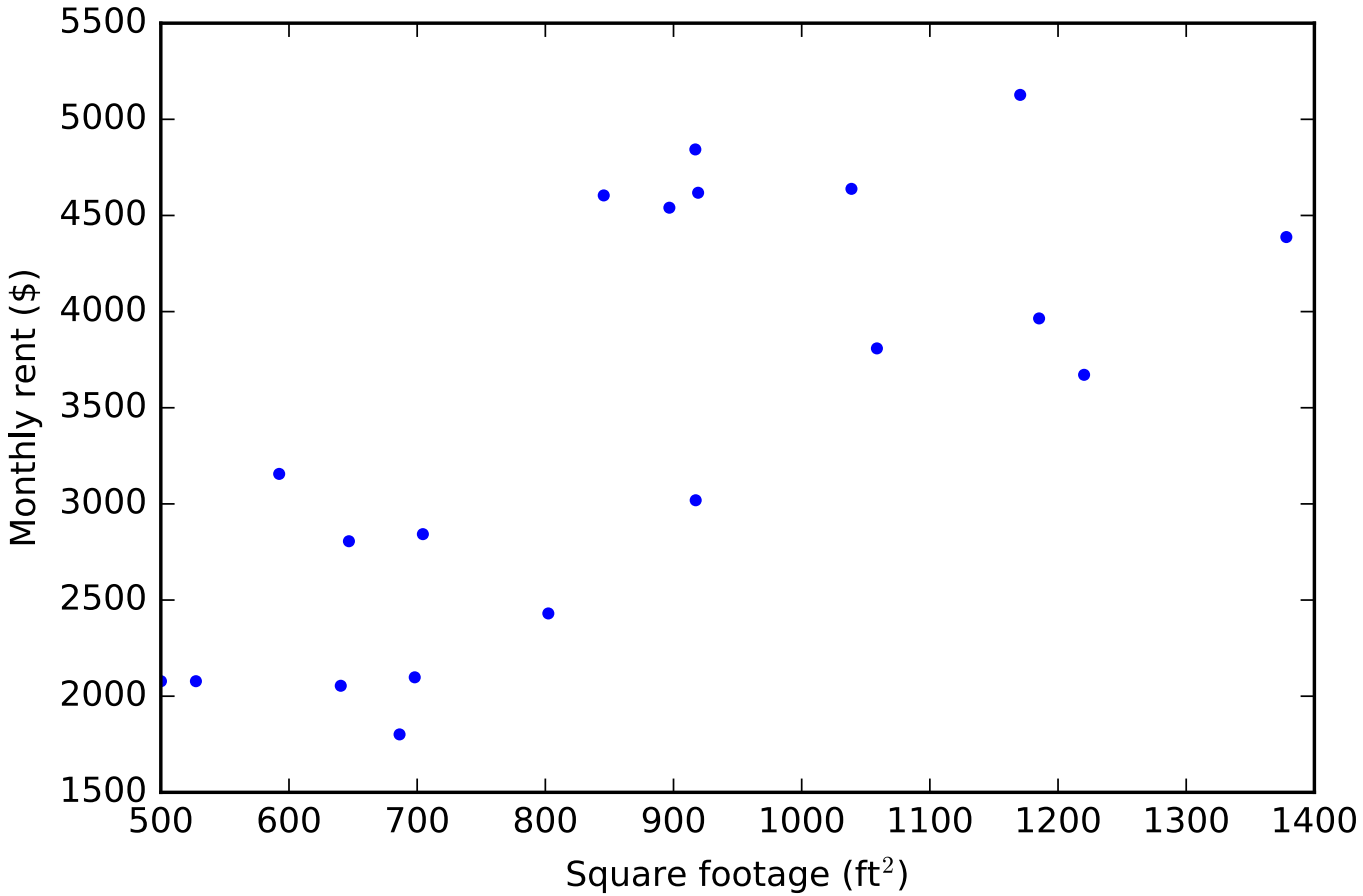
$f(x^{(i)}) \sim y^{(i)}$

to be specified

An example of supervised learning

Regression $y^{(i)} \in \mathbb{R}$

Simple problem: Regression



output = rent

How should we model this data?

→ input square footage

- Inputs, **x**? Outputs, **y**?
- What model should we use?
- How do we assess how good our model is?

GOAL: Learn $f(\cdot)$

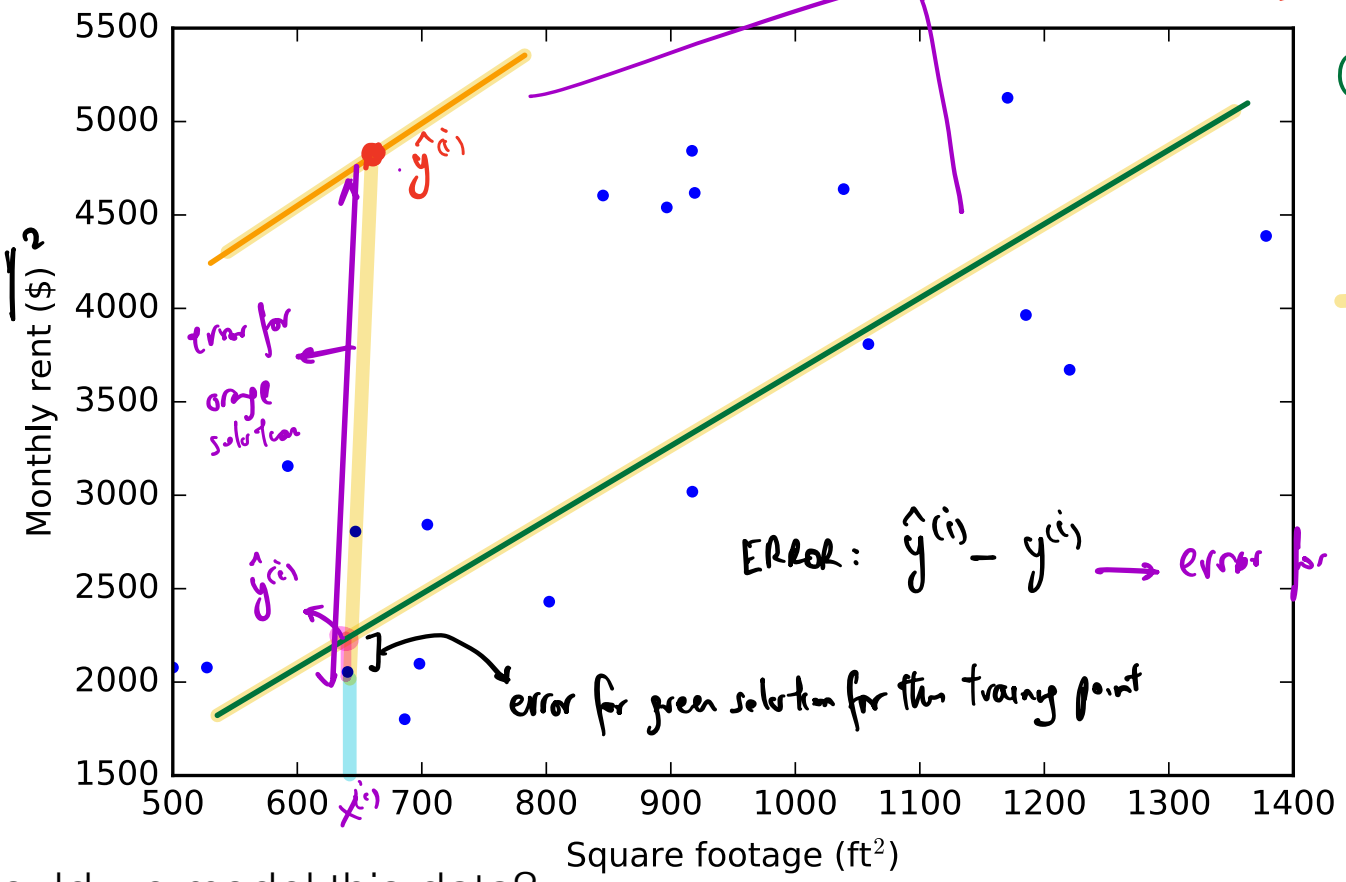
$f(\text{sq footage}) \sim \text{rent}$

input, output, to specify

An example of supervised learning

MODEL: LINEAR function

$y = f(x) = ax + b$



How should we model this data?

- Inputs, **x**? Outputs, **y**?
- What model should we use?
- How do we assess how good our model is?

Observation:

- Want to find error (avg) all D
- Magnitude of error matters in this case not the sign of error

An example of supervised learning

A simple linear example:

$\hat{y} = ax + b = [a, b] \begin{bmatrix} x \\ 1 \end{bmatrix}$, $\hat{x} = \begin{bmatrix} x \\ 1 \end{bmatrix}$, $\theta = \begin{bmatrix} a \\ b \end{bmatrix}$, θ^T

- Model:

$$\hat{y} = ax + b \longrightarrow \text{LINEAR MODEL}$$
$$= \theta^T \hat{\mathbf{x}}$$

$\theta \rightarrow$ vector of (unknown) parameters to be found

$y = f(x) = a_3 x^3 + a_2 x^2 + a_1 x + b$

$= [a_3\ a_2\ a_1\ b] \begin{bmatrix} x^3 \\ x^2 \\ x \\ 1 \end{bmatrix}$

- Cost function:

$= \theta^T \hat{x}$

$$\mathcal{L}(\theta) = \frac{1}{2N} \sum_{i=1}^{N} (y^{(i)} - \hat{y}^{(i)})^2$$

$N \rightarrow$ normalization constant

$\theta \in \mathbb{R}^4$

representing all polynomials of order 3

$$= \frac{1}{2N} \sum_{i=1}^{N} (y^{(i)} - \theta^T \hat{\mathbf{x}}^{(i)})^2$$

↳ Definition: This notation can seamlessly handle polynomial regression

How do we learn the parameters of this model?

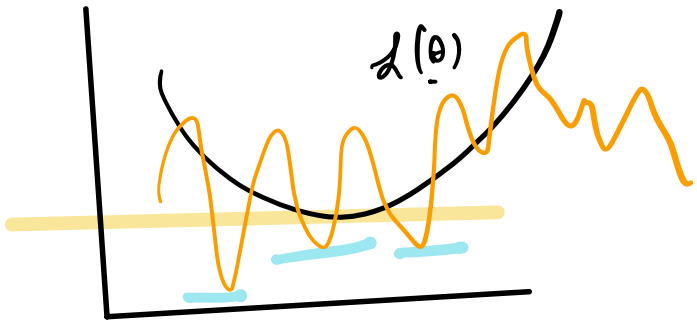
QUESTION: Can we find the model with the lowest loss

$$\mathcal{L}(\theta) = \frac{1}{2N} \sum_{i=1}^{N} |y^{(i)} - \hat{y}^{(i)}|^2, \quad \hat{y}^{(i)} = \theta^T \hat{x}^{(i)}$$

An example of supervised learning

Construct it as an optimization problem.

Our goal is to choose the parameters to make the loss as small as possible.



Thus our strategy to find the best θ is to:

- Calculate $\dfrac{d\mathcal{L}}{d\theta}$
- Solve for θ such that $\dfrac{\partial \mathcal{L}}{\partial \theta} = 0$

vector

$$\mathcal{L}(\underline{\theta}) = \frac{1}{2N} \sum_{i=1}^{N} \left| y^{(i)} - \underline{\theta}^T \underline{\hat{x}}^{(i)} \right|^2$$

quadratic (convex) loss function

However, θ is a vector, so how do we take derivatives with respect to it?



Aside: vector and matrix derivatives

In machine learning, we often take derivatives with respect to vectors and matrices.

These are typically called *gradients*, and in this class we'll use the following notation to denote derivatives with respect to vectors and matrices.

In this class, we will use both the differentiation operator ∂ and ∇ to denote derivatives. If y is a scalar, and $\mathbf{x}$ is a vector, then the gradient of y with respect to $\mathbf{x}$ is denoted as both:

$$\frac{\partial y}{\partial \mathbf{x}} \quad \text{and} \quad \nabla_{\mathbf{x}} y$$

The gradient of y with respect to $\mathbf{x}$ is itself a vector with the same dimensionality as $\mathbf{x}$.

Aside: vector and matrix derivatives

The gradient

$y = \text{scalar} \in \mathbb{R}$ $\quad \underline{x} \in \mathbb{R}^n \quad \underline{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$

The gradient generalizes the scalar derivative to multiple dimensions. Suppose $f : \mathbb{R}^n \to \mathbb{R}$ transforms a vector $\mathbf{x} \in \mathbb{R}^n$ to a scalar. If $y = f(\mathbf{x})$, then the gradient is:

$$\nabla_{\mathbf{x}} y = \begin{bmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \\ \vdots \\ \frac{\partial y}{\partial x_n} \end{bmatrix}$$

$\nabla_{\underline{x}}(y) = \begin{bmatrix} 1 \\ 0.5 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$

$\Rightarrow \frac{\partial y}{\partial x_1} = 1, \quad \frac{\partial y}{\partial x_2} = 0.5, \quad \partial y / \partial x_3 = 0$ etc

Wiggle of $x_1 \sim x_1 + \Delta x_1$

$\Delta y_1 \sim \frac{\partial y}{\partial x_1} \Delta x_1$

In general $\Delta y_i \sim \frac{\partial y}{\partial x_i} \Delta x_i$

Aside: vector and matrix derivatives

In other words, the gradient is:

- A *vector* that is the same size as $\mathbf{x}$, i.e., if $\mathbf{x} \in \mathbb{R}^n$ then $\nabla_{\mathbf{x}} y \in \mathbb{R}^n$.
- Each dimension of $\nabla_{\mathbf{x}} y$ tells us how small changes in $\mathbf{x}$ in that dimension affect y. i.e., changing the ith dimension of $\mathbf{x}$ by a small amount, Δx_i, will change y by

$$\frac{\partial y}{\partial x_i} \Delta x_i$$

i^{th} Component

$$(\nabla_{\mathbf{x}} y)_i \, \Delta x_i$$

We may also denote this as:

$$\sum_{i=1}^{n} (\nabla_x y)_i \, \Delta x_i$$

$$(\nabla_x y)_i = \frac{\partial y}{\partial x_i}$$

$$\Delta \underline{x} = \begin{bmatrix} \Delta x_1 \\ \Delta x_2 \\ \vdots \\ \Delta x_n \end{bmatrix}$$

$$\Delta y \approx \frac{\partial y}{\partial x_1} \Delta x_1 + \frac{\partial y}{\partial x_2} \Delta x_2 + \cdots + \frac{\partial y}{\partial x_n} \Delta x_n$$

$$= \begin{bmatrix} \frac{\partial y}{\partial x_1} & \cdots & \frac{\partial y}{\partial x_n} \end{bmatrix} \begin{bmatrix} \Delta x_1 \\ \vdots \\ \Delta x_n \end{bmatrix}$$

$$= (\nabla_x y)^T \Delta \underline{x}$$

Aside: vector and matrix derivatives

Example: derivative with respect to a vector

Let $f(\mathbf{x}) = \theta^T \mathbf{x}$. What is $\nabla_{\mathbf{x}} f(\mathbf{x})$?

$\underline{\theta} = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$, $\underline{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$

$\underline{\theta} \in \mathbb{R}^n$, $\underline{x} \in \mathbb{R}^n$

$y = \underline{\theta}^T \underline{x}$, $y \in \mathbb{R}$

$y = \underline{\theta}^T \underline{x}$ $\quad \nabla_{\underline{x}} y$ to compute

$$y = \underline{\theta}^T \underline{x} = \sum_{i=1}^{n} \theta_i x_i = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$\nabla_{\underline{x}} y = \begin{bmatrix} \partial y / \partial x_1 \\ \vdots \\ \partial y / \partial x_n \end{bmatrix} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} = \underline{\theta}$$

$$\nabla_{\underline{x}} y = \underline{\theta}$$

Similarly $\nabla_{\underline{\theta}} y = \underline{x}$

$n = 1 \rightarrow y = \theta x$ $\quad \frac{\partial y}{\partial x} = \theta$, $\frac{\partial y}{\partial \theta} = x$

Aside: vector and matrix derivatives

Example: derivative with respect to a vector

Let $f(\mathbf{x}) = \mathbf{x}^T \mathbf{A}\mathbf{x}$. What is $\nabla_{\mathbf{x}} f(\mathbf{x})$?

$\underline{x} \in \mathbb{R}^n$, $\underline{\underline{A}} \in \mathbb{R}^{n \times n}$ $\quad \underline{\underline{A}} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \\ a_{n1} & \cdots & a_{nn} \end{bmatrix}$, $\underline{\underline{A}}\,\underline{x} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \\ a_{n1} & \cdots & a_{nn} \end{bmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}$

$$\underline{x}^T \underline{\underline{A}}\,\underline{x} = \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix} \begin{bmatrix} \sum a_{1j} x_j \\ \vdots \\ \sum a_{nj} x_j \end{bmatrix} = \begin{bmatrix} \sum a_{1j} x_j \\ \vdots \\ \sum a_{nj} x_j \end{bmatrix}$$

$$= x_1 \sum_{j=1}^{n} a_{1j} x_j + x_2 \sum_{j=1}^{n} a_{2j} x_j + \cdots x_n \sum_{j=1}^{n} a_{nj} x_j$$

$$f(x) = \underline{x}^T \underline{\underline{A}}\,\underline{x} = \sum_{i=1}^{n} \sum_{j=1}^{n} x_i a_{ij} x_j$$

$\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \cdots$

Aside: vector and matrix derivatives

$$f(x) = \underline{x}^T \underline{\underline{A}}\, \underline{x} = \sum_{i=1}^{n} \sum_{j=1}^{n} x_i a_{ij} x_j \qquad \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \cdots \frac{\partial f}{\partial x_n}$$

$$\frac{\partial f}{\partial x_1} = \frac{\partial}{\partial x_1}\left[x_1^2 a_{11} + \sum_{j=2}^{n} x_1 a_{1j} x_j + \sum_{i=2}^{n} x_i a_{i1} x_1 \right] = 2 x_1 a_{11} + \sum_{j=2}^{n} a_{1j} x_j + \sum_{i=2}^{n} a_{i1} x_i$$

$$= \sum_{j=1}^{n} a_{1j} x_j + \sum_{i=1}^{n} a_{i1} x_i = (\underline{\underline{A}}\,\underline{x})_1 + (\underline{\underline{A}}^T \underline{x})_1$$

In general $\frac{\partial f}{\partial x_i} = (\underline{\underline{A}}\,\underline{x})_i + (\underline{\underline{A}}^T \underline{x})_i$

$$\frac{\partial f}{\partial \underline{x}} = \begin{bmatrix} \partial f/\partial x_1 \\ \vdots \\ \partial f/\partial x_n \end{bmatrix} = \underline{\underline{A}}\,\underline{x} + \underline{\underline{A}}^T \underline{x} = (\underline{\underline{A}} + \underline{\underline{A}}^T)\,\underline{x}$$

if $\underline{\underline{A}} = \underline{\underline{A}}^T$ (ie symmetric matrix)

then $\frac{\partial f}{\partial \underline{x}} = 2\,\underline{\underline{A}}\,\underline{x}$

Sanity check

$n = 1$

$f(x) = a x^2$

$\frac{\partial f}{\partial x} = 2ax$

Aside: vector and matrix derivatives

First, we note that $\mathbf{x}^T\mathbf{A}\mathbf{x} = \sum_i \sum_j a_{ij} x_i x_j$. Then, we have

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} 2a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n + a_{21}x_2 + \cdots + a_{n1}x_n \\ 2a_{22}x_2 + a_{21}x_1 + \cdots + a_{2n}x_n + a_{12}x_1 + \cdots + a_{n2}x_n \\ \vdots \\ 2a_{nn}x_n + a_{n1}x_1 + \cdots + a_{n,n-1}x_{n-1} + a_{1n}x_1 + \cdots + a_{n-1,n}x_{n-1} \end{bmatrix}$$

$$= \mathbf{A}\mathbf{x} + \mathbf{A}^T\mathbf{x}$$

↳ We just derived this

Matrix derivatives

$\sum_{j=1}^{m}\sum_{i=1}^{n} z_i a_{ij} x_j$ → DO AS EXERCISE

- Gradient of a scalar with respect to a matrix

$$y = \underline{z}^T \underline{\underline{A}}\, \underline{x}, \quad y \in \mathbb{R}, \quad \underline{z} \in \mathbb{R}^m, \quad \underline{x} \in \mathbb{R}^n, \quad \underline{\underline{A}} \in \mathbb{R}^{m \times n}$$

$\underline{z}^T$: $1 \times m$; $\underline{\underline{A}}$: ? $m \times n$; $\underline{x}$: $n \times 1$

$$\frac{\partial y}{\partial \underline{\underline{A}}} = \nabla_{\underline{\underline{A}}} y = \begin{bmatrix} \partial y/\partial a_{11} & \cdots & \partial y/\partial a_{1n} \\ \vdots & & \\ \partial y/\partial a_{m1} & \cdots & \partial y/\partial a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

→ Denominator form → we will use this form

Numerator form → $\begin{bmatrix} \partial y/\partial a_{11} & \cdots & \partial y/\partial a_{m1} \\ \vdots & & \\ \partial y/\partial a_{1n} & \cdots & \partial y/\partial a_{mn} \end{bmatrix}$

Generalizing to derivative of a vector with respect to a matrix

$u \in \mathbb{R}^p$ $\quad \underline{\underline{A}} \in \mathbb{R}^{m \times n}$ then denominator form of $\frac{\partial u}{\partial \underline{\underline{A}}} \in \mathbb{R}^{m \times n \times p}$ → tensor.

Back to our supervised learning example

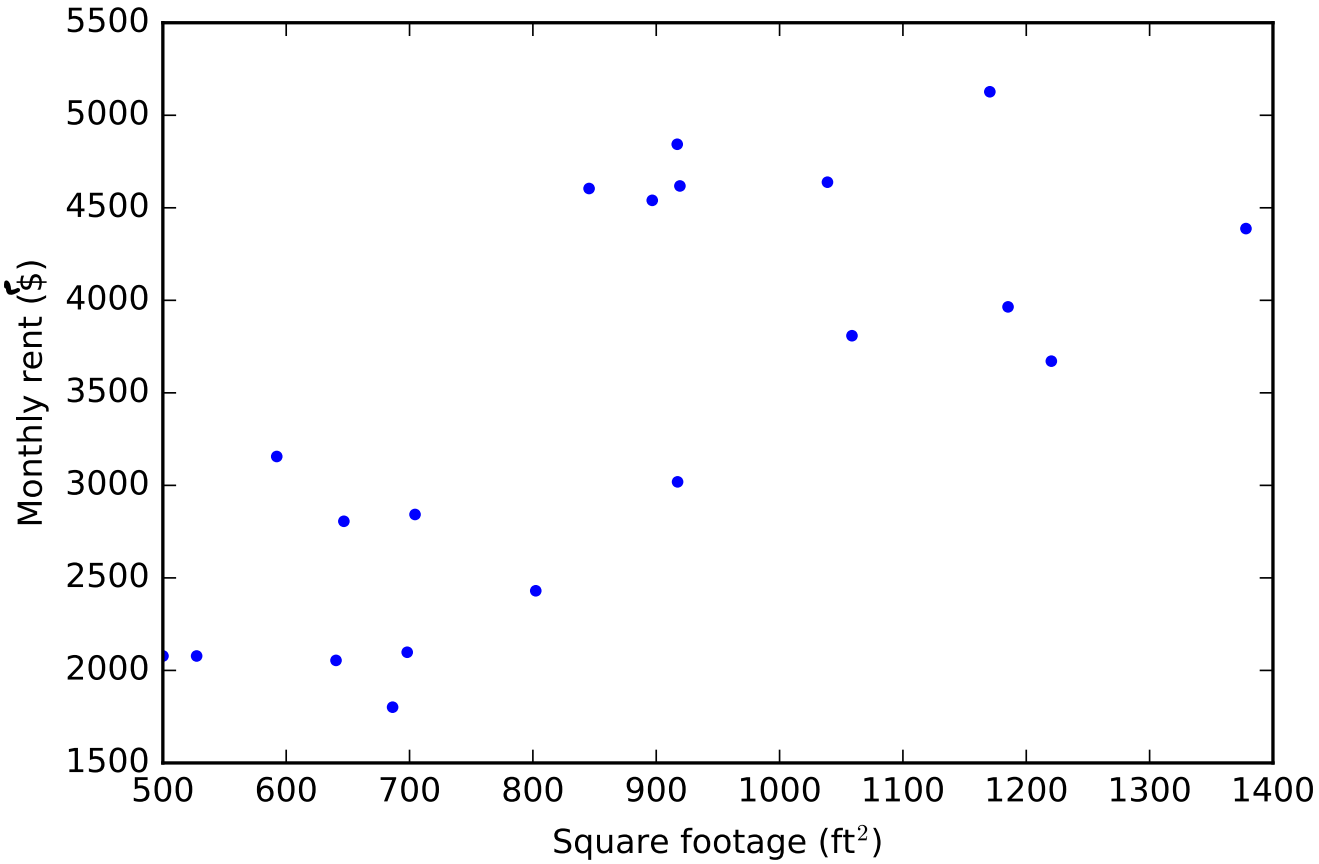
Back to why we took this detour.

$$\mathcal{L}(\theta) = \frac{1}{2N} \sum_{i=1}^{n} |y^{(i)} - \hat{y}^{(i)}|^2$$

$$\hat{y}^{(i)} := \theta^T \hat{x}^{(i)}$$

GOAL

$$\min_{\theta} \mathcal{L}(\theta)$$



To do this we want to find

$$\frac{\partial \mathcal{L}}{\partial \theta} = 0$$

Thus our strategy to find the best θ is to:

- Calculate

$$\frac{d\mathcal{L}}{d\theta}$$

- Solve for θ such that

$$\frac{\partial \mathcal{L}}{\partial \theta} = 0$$

Back to our supervised learning example

Re-writing the cost function, we have: $\theta \in \mathbb{R}^n$, $x^{(i)} \in \mathbb{R}^n$

$$\mathcal{L} = \frac{1}{2N}\sum_{i=1}^{N}(y^{(i)} - \theta^T \hat{\mathbf{x}}^{(i)})^2$$

If $S \in \mathbb{R}$, $S^T = S$

Fact: $(ABC)^T = C^T B^T A^T$, $(\theta^T \hat{x})^T = (\hat{x}^T \theta)$

$$= \frac{1}{2N}\sum_{i=1}^{N}\left[y^{(i)} - \theta^T \hat{x}^{(i)}\right]\left[y^{(i)} - \theta^T \hat{x}^{(i)}\right]$$

$$= \frac{1}{2N}\sum_{i=1}^{N}\left[y^{(i)} - \hat{x}^{(i)T}\theta\right]\left[y^{(i)} - \hat{x}^{(i)T}\theta\right]$$

Vectorization

$$= \frac{1}{2N}\begin{bmatrix} y^{(1)} - \hat{x}^{(1)T}\theta \\ \vdots \\ y^{(N)} - \hat{x}^{(N)T}\theta \end{bmatrix}^T \begin{bmatrix} y^{(1)} - \hat{x}^{(1)T}\theta \\ \vdots \\ y^{(N)} - \hat{x}^{(N)T}\theta \end{bmatrix}$$

$$X = \begin{bmatrix} \hat{x}^{(1)T} \\ \vdots \\ \hat{x}^{(N)T} \end{bmatrix}, \quad N \times n, \qquad y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{bmatrix}$$

$$= \frac{1}{2N}(y - X\theta)^T(y - X\theta)$$

ASIDE

$$\sum c_i d_i = \begin{bmatrix} c_1 \\ \vdots \\ c_n \end{bmatrix}^T \begin{bmatrix} d_1 \\ \vdots \\ d_n \end{bmatrix}$$

$$c_i = y^{(i)} - \hat{x}^{(i)T}\theta$$

$$d_i = c_i$$

$$\begin{bmatrix} y^{(1)} - \hat{x}^{(1)T}\theta \\ \vdots \\ y^{(N)} - \hat{x}^{(N)T}\theta \end{bmatrix} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{bmatrix} - X\theta = y - X\theta$$

Back to our supervised learning example

Let's now take derivatives:

$$(\underline{y} - \underline{\underline{X}}\,\underline{\theta})^T (\underline{y} - \underline{\underline{X}}\,\underline{\theta}) = \underline{y}^T\underline{y} + \underline{\theta}^T\underline{\underline{X}}^T\underline{\underline{X}}\,\underline{\theta} - 2\,\underline{y}^T\underline{\underline{X}}\,\underline{\theta}$$

$$J(\theta) = \frac{1}{2N}\left(Y^TY - 2Y^T\mathbf{X}\theta + \theta^T\mathbf{X}^T\mathbf{X}\theta\right)$$

$\frac{\partial J}{\partial \underline{\theta}} = \underline{0}$ to find optimal $\underline{\theta}$

$$= \frac{1}{2N}\left[0 - 2\,\underline{\underline{X}}^T\underline{y} + 2\,\underline{\underline{X}}^T\underline{\underline{X}}\,\underline{\theta}\right] = \underline{0}$$

$$\Rightarrow \quad \underline{\underline{X}}^T\underline{\underline{X}}\,\underline{\theta} = \underline{\underline{X}}^T\underline{y} \quad \text{or} \quad \boxed{\underline{\theta} = (\underline{\underline{X}}^T\underline{\underline{X}})^{-1}\underline{\underline{X}}^T\underline{y}}$$

$f(x) = x^T\theta$

$\frac{\partial f}{\partial \underline{\theta}} = \underline{x}$

$f(x) = \underline{\theta}^T\underline{\underline{A}}\,\underline{\theta}$

$\frac{\partial f}{\partial \underline{\theta}} = (\underline{\underline{A}} + \underline{\underline{A}}^T)\,\underline{\theta}$



Back to our supervised learning example

This solution is called least-squares, and it appears in a variety of linear applications.

$$\theta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T Y$$

Back to our supervised learning example

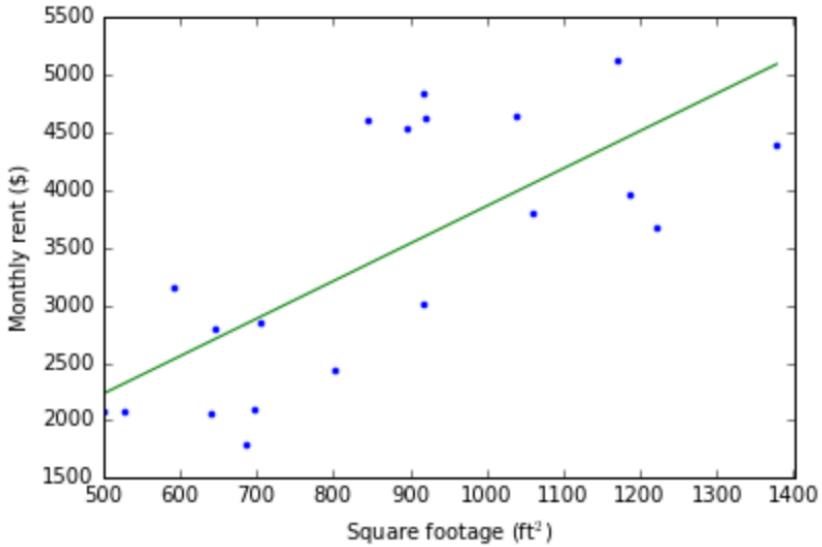
```
# Given paired data, with x being square footages and y being monthly rents
# Fit a linear model
xhat = np.vstack((x, np.ones_like(x)))
theta_lin = np.linalg.inv(xhat.dot(xhat.T)).dot(xhat.dot(y))

# Plot the data
f = plt.figure()
ax = f.gca()
ax.plot(x, y, '.')
ax.set_xlabel('Square footage (ft$^2$)')
ax.set_ylabel('Monthly rent ($)')
f.savefig('ml-basics_rent.pdf')

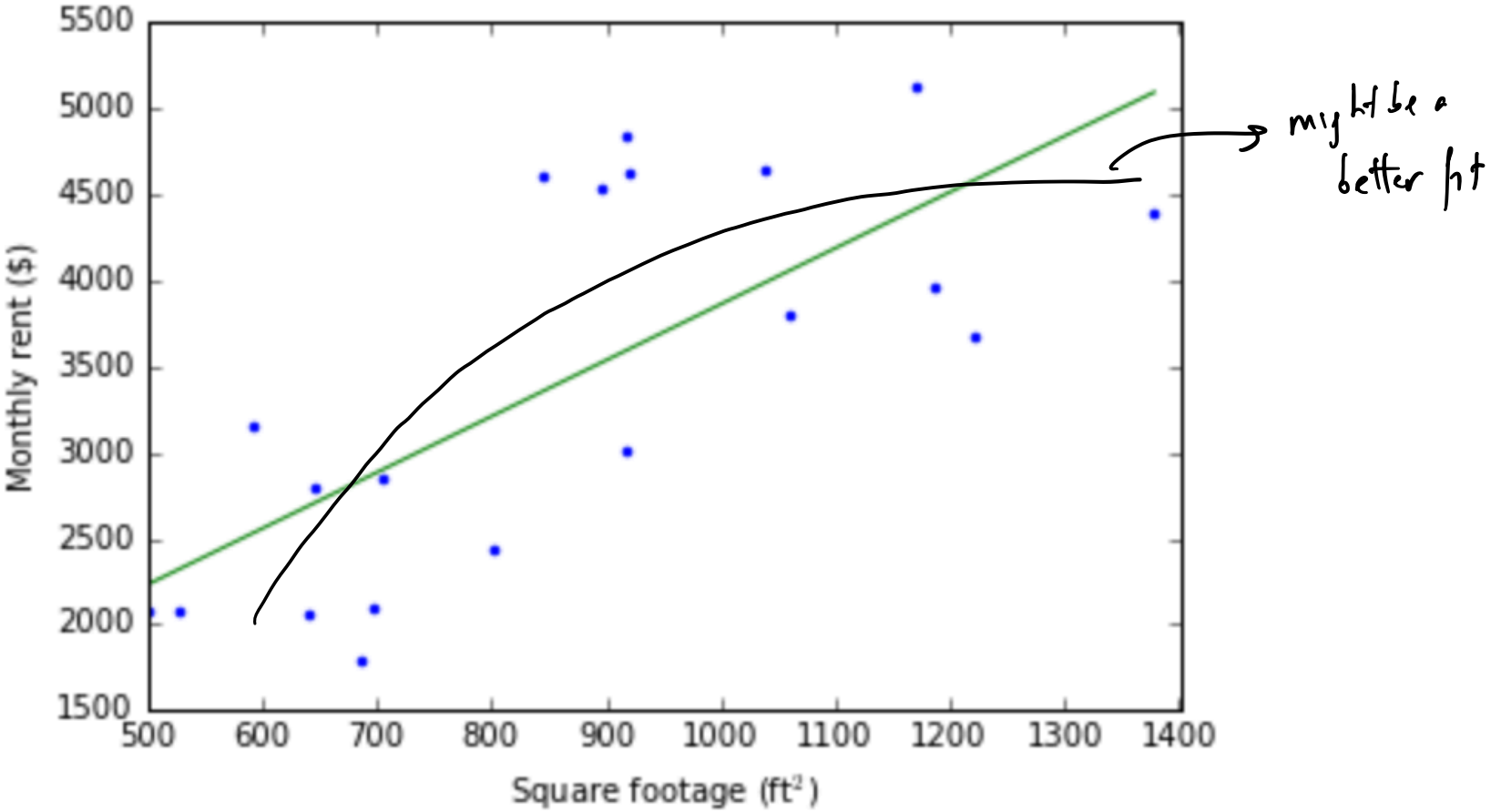
# Plot the regression line
xs = np.linspace(min(x), max(x),50)
xs = np.vstack((xs, np.ones_like(xs)))
plt.plot(xs[0,:], theta_lin.dot(xs))

print theta_lin
```

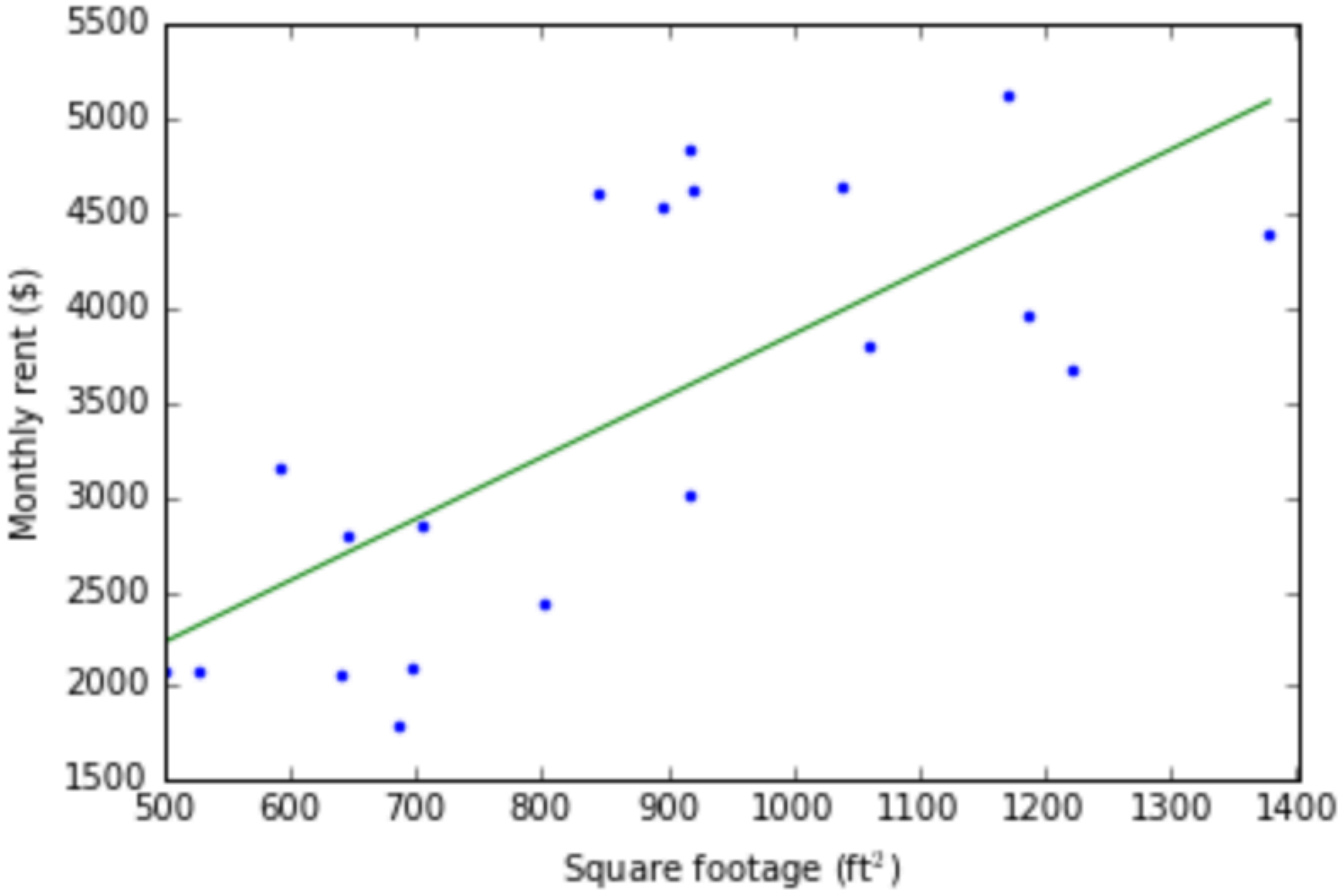
```
[    3.25900793   601.85544895]
```



Can we do better?



Question



Question: Does our current least-squares formula allow for learning nonlinear polynomial fits, e.g.,

$$y = b + a_1 x_1 + a_2 x^2 + \cdots + a_n x^n$$

More generally…

With our problem setup it is straightforward to generalize to higher degree polynomial models e.g.,

$$y = b + a_1 x + a_2 x^2 + \ldots + a_n x^n = \theta^\top \hat{x}$$

where $\theta = \begin{bmatrix} b \\ a_1 \\ \vdots \\ a_n \end{bmatrix}$ and $\hat{x} = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^n \end{bmatrix}$

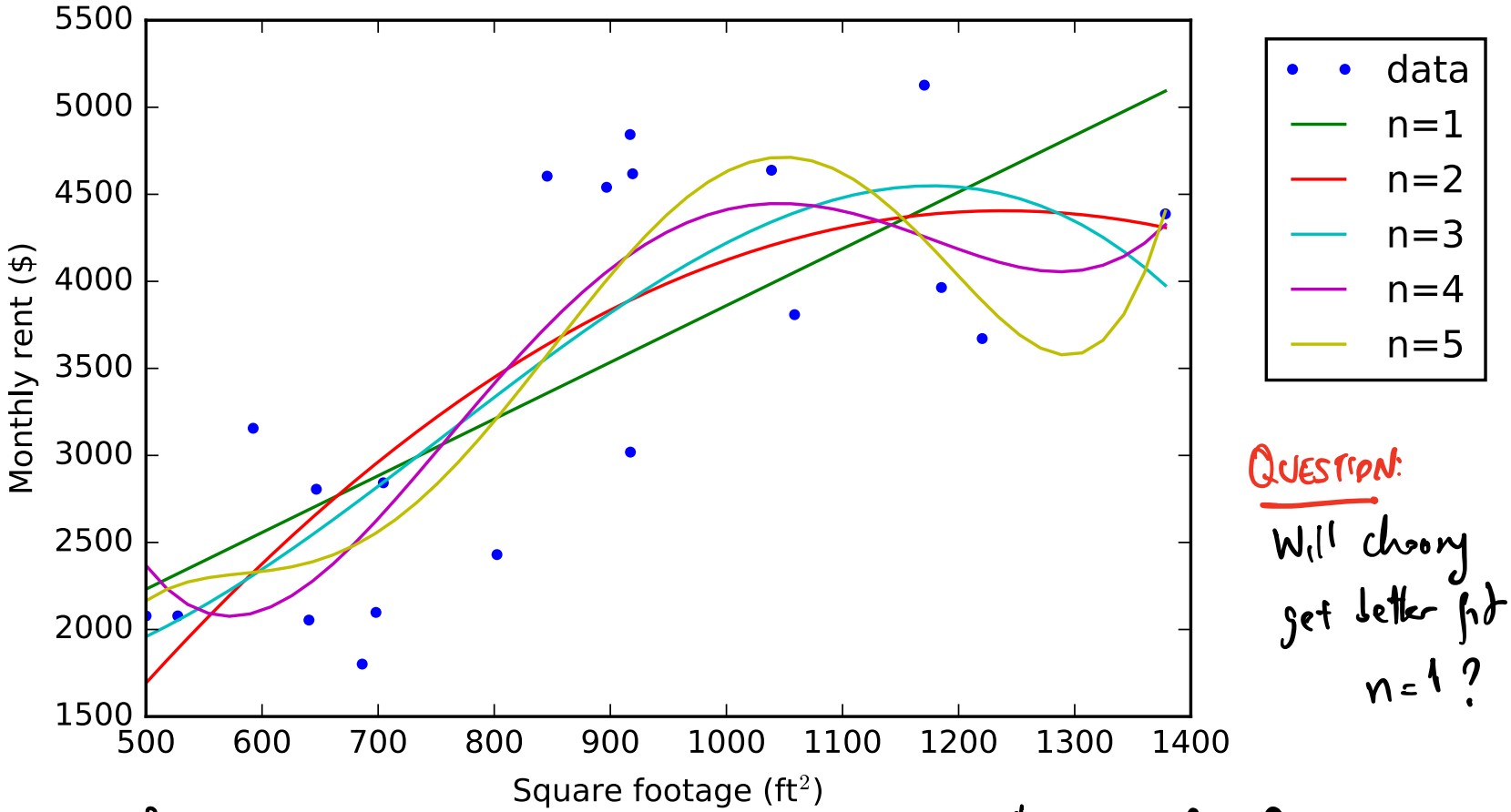
parameters

data representation

$\hat{X}^\top$ = known and constructed from the data

$\underline{y} = \underline{\theta}^\top \hat{\underline{X}}$ then $\underline{\theta}^* = (\hat{\underline{X}}^\top \underline{\underline{X}})^{-1} \hat{\underline{X}}^\top \underline{y}$

More generally…



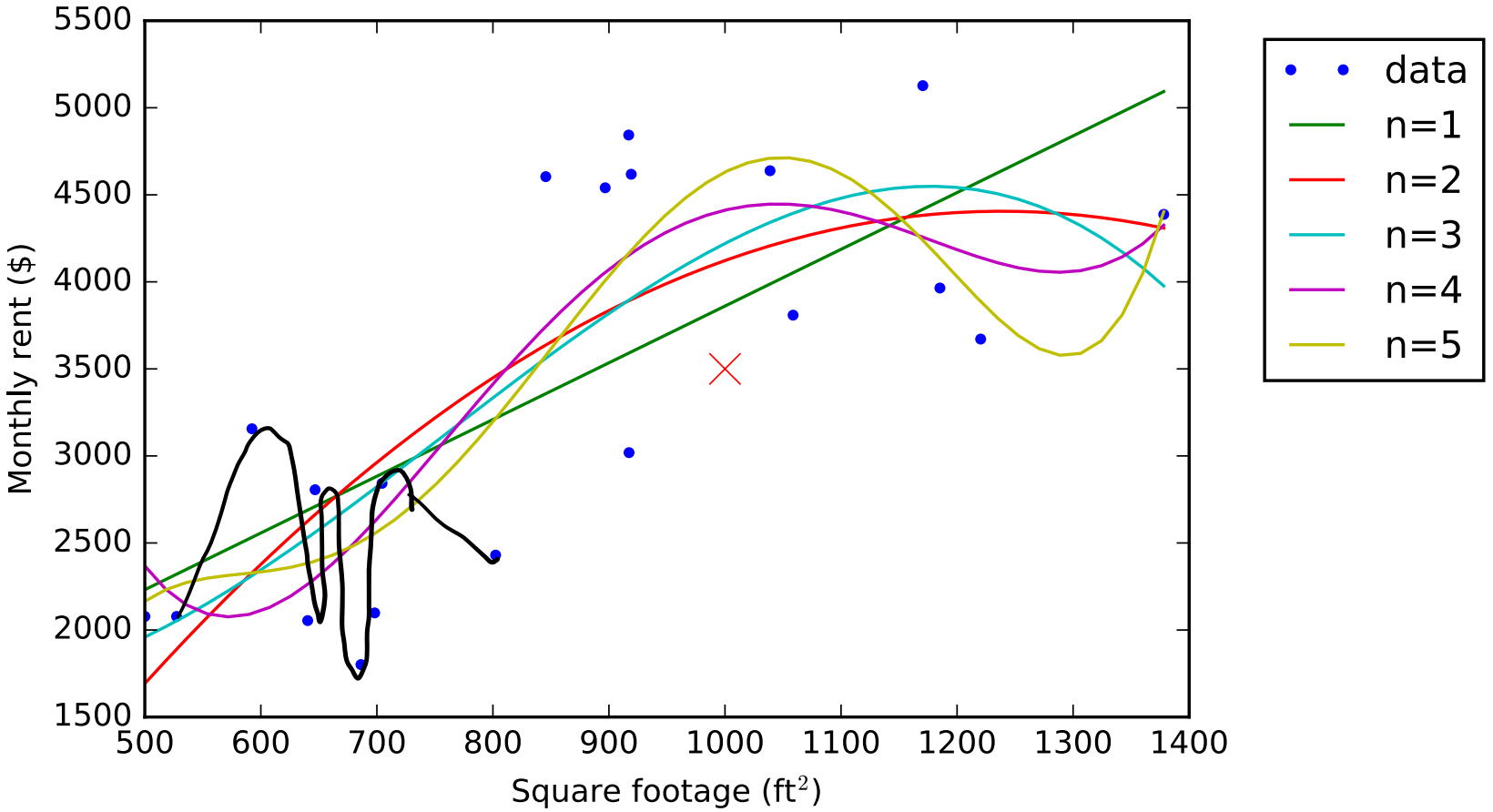
QUESTION:
Will choosing $n=3$ get better fit than $n=1$?

$y = a_3x^3 + a_2x^2 + a_1x + b$, $y = ax + b$ → get this by setting $a_3 = a_2 = 0$.

Now, a higher degree polynomial will *always* fit the **provided** data better. (Why)?

$J(\underline{\theta}) = \frac{1}{2N}\sum (y^{(i)} - \hat{y}^{(i)})^2$, $\hat{y}^{(i)} \rightarrow n^{th}$ order polynomial.

Now, let us say a new house pops up for rent…



In this scenario, the linear model best predicts the price of the new house ('x')